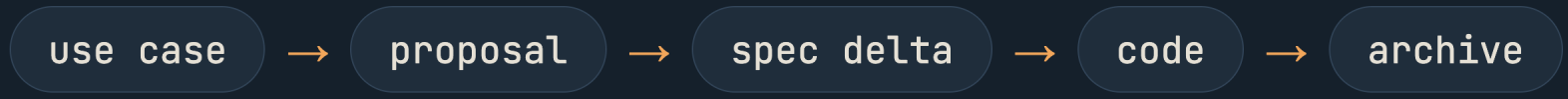
ENGINEERING TALK

Spec first, then agents

OpenSpec, and a multi-agent pipeline that drives it from use case to merged code. Includes what it really costs.



Paul Rule · October 2026

PART 1

OpenSpec

Agree on the behaviour in writing, then build it.

THE PROBLEM

Chat-driven coding loses the “why”

Intent lives in prompts

The requirement is scattered across a chat transcript that nobody reads again.

Nothing to check against

Reviewers see the code, but not the behaviour it was supposed to deliver.

Every session starts cold

The next agent, or the next person, can't see what was decided or why.

OpenSpec moves the agreement out of the chat and into a file, before any code is written.

WHAT IT IS

Specs are the truth; changes are deltas

`openspec/specs/`

What the system does today, written as requirements and scenarios.

`openspec/changes/<name>/`

A proposed change: why, how, and a delta against those specs.

`propose`
Write the proposal, design, spec delta and tasks

→

`apply`
Implement tasks.md against the delta

→

`verify`
Check the code against every requirement

→

`archive`
Fold the delta into the living specs

It's plain Markdown in the repo, driven by slash commands in whichever coding agent you use.

ANATOMY OF A CHANGE

Four kinds of file capture the agreement

```
add-movie-search/
├─ proposal.md
├─ design.md
├─ specs/
│  ├─ catalog/movies/spec.md
│  ├─ platform/collection-paging/…
│  └─ platform/uniform-responses/…
└─ tasks.md
```

proposal: why, what changes, non-goals

design: decisions D1–D6, risks, open questions

specs/: ADDED / MODIFIED requirements + scenarios

tasks: 20 checkboxes, from contract to code to tests

```
specs/catalog/movies/spec.md (excerpt)
## ADDED Requirements
### Requirement: Search and browse movies
The service SHALL offer GET /api/v1/movies. Every query
parameter is optional …

#### Scenario: Repeated sort is refused
- WHEN a client sends GET /api/v1/movies?
sort=title&sort=rating
- THEN the response status is 400, code is BAD_REQUEST,
and detail contains 'sort'
```

PROJECT CONTEXT

config.yaml injects your house rules

```yaml
schema: spec-driven
context: |
  Contract-first (API-first):
    - Every HTTP API is defined in an OpenAPI 3.1 spec BEFORE any
controller code.
  Testing: … ALL database tests use Testcontainers against real
Postgres — never H2.
rules:
  proposal:
    - Always include a "Non-goals" section.
  design:
    - "Include the DB migration plan (Flyway) and rollback
consideration."
  tasks:
    - "Each task should be completable in under ~2 hours."
```

context

Shown to the AI whenever it writes an artifact.

rules

A checklist per artifact: proposal, design, specs, tasks.

operations

Guidance for apply and archive, e.g. run codegen before writing controllers.

Excerpt from openspec/config.yaml in this repo

HOW TO USE IT

A handful of commands

Command	When	What you get
npm i -g @fission-ai/openspec	once	the CLI
openspec init	once per repo	openspec/ and agent slash commands
/opsx:explore	idea is fuzzy	a thinking partner; no code yet
/opsx:propose "<idea>"	start a change	proposal, design, spec delta, tasks
openspec validate <change> --strict	before review	structural check of the delta
/opsx:apply <change>	plan approved	code, ticking off tasks.md
/opsx:verify <change>	code done	code checked against the artifacts
/opsx:archive <change>	merged	delta folded into openspec/specs/

WHAT YOU END UP WITH

Living specs, plus a history you can read

`openspec/specs/`

What the system does now. Six capabilities in this repo:

```
catalog/movies
platform/availability-check
platform/collection-paging
platform/interface-description
platform/runtime-modes
platform/uniform-responses
```

`openspec/changes/archive/`

Why each change happened, and what was decided:

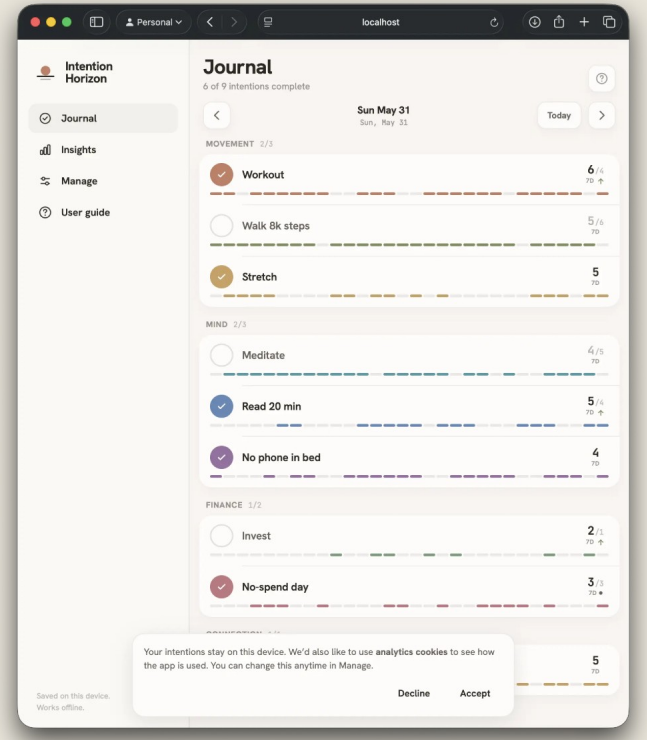
```
2026-09-27-add-platform-foundation
2026-09-28-add-movie-details
2026-09-28-add-movie-search
```

Each one keeps its proposal, design, delta and tasks.

The next person, or agent, starts from the specs instead of a chat log.

EXAMPLE

Intention Horizon, built with OpenSpec



A local-first habit tracker: a React PWA on IndexedDB, with no backend. I built it on my own with Claude Code and OpenSpec. **No agent pipeline.**

27 archived changes

17 capability specs

2 months, Jun 4 to Aug 3

Try it: intention.vamonossoftware.com

INTENTION HORIZON · CHANGES/ARCHIVE

Each change became a named unit

Features · 12

add-url-routing
flexible-completion-target
add-data-source-toggle
default-real-gate-mock
insights-sort-recent-first
rename-today-to-journal
add-analytics-consent
add-guide-starter-examples
csv-import
anchor-journal-to-today
add-update-notification
show-git-version

Tests & quality · 8

add-e2e-playwright-screenplay
refactor-e2e-serenity-screenplay
add-serenity-reports
add-e2e-persistence-reload
add-e2e-categories-targets-analytics
fix-e2e-consent-overlay
add-vitest
vitest-store-coverage

Platform, CI & fixes · 7

migrate-to-typescript
ci-run-e2e-on-build
bump-deprecated-github-actions
add-screenshot-fixture-script
fix-dialog-fullscreen-scrim
fix-dialog-overlay-styling
fix-journal-scroll-reset

TRADE-OFFS

What OpenSpec gives you, and what it costs

Gives you

- Intent is reviewed before any code exists
- Specs outlive the chat: lasting behaviour you can diff
- Small, named changes, each traceable to a reason
- Plain Markdown that any agent or person can read
- House rules applied automatically via config.yaml

Costs you

- Ceremony: four artifacts, even for a one-line fix
- More tokens and more wall time per change
- Specs drift unless every change is archived
- A bad plan gets built faithfully
- Nothing independently checks the plan

That last gap is what Part 2 is about.

PART 2

The agent pipeline

Six roles, two human gates, and one OpenSpec change per use case.

WHY AGENTS

Why put agents on top of OpenSpec?

Separate contexts

Planner, builder and reviewers each run in their own context. No agent marks its own work.

Independent gates

A read-only spec-reviewer checks the plan. QA and a senior reviewer check the code.

Roles, not tiers

Every role runs on Opus. The roles differ in tools, context and job, not in model.

“OpenSpec owns the workflow mechanics; agents own judgment + standards.”

The design principle, from the repo README

STRUCTURE

Six agents, each one a Markdown file

Agent	Model	Writes?	Job	OpenSpec
use-case-writer	Opus	use-cases/	Rough idea → use case, grounded in domain/	—
architect	Opus	plan files	Use case → proposal, design, delta, tasks	propose
spec-reviewer	Opus	read-only	Plan gate: standards and design soundness	validate
junior-dev	Opus	yes	Works through tasks.md: code and tests, in plan	apply
qa	Opus	tests	Maps each requirement to a test; runs suite	verify
senior-dev	Opus	fixes in place	Reviews the diff and fixes what it finds; can implement instead	apply, if implementing

Plus an orchestrator, /sdlc-pipeline:build-use-case. It sequences the agents, passes the change name between them and enforces the gates. It does none of the phase work itself.

agents/*.md in the plugin, each with frontmatter for name, model and tools

PACKAGED

Now a Claude Code plugin, for any repo and stack

`PRD` → `init` → `write-domain` → `write-standards` → `plan-use-cases` → `write-use-case` → `build-use-case`

Set a project up once, in this order. Then write and build one use case at a time (the dark pair).

Profile

.claude/sdlc-profile.md holds the stack: verify command, paths, review checklists. The agents stay stack-agnostic.

Backlog

plan-use-cases writes an MVP-first BACKLOG.md. Each archived change ticks its entry.

Records

Every run leaves a retrospective, a session report with its summary.json, and an updated ledger.

INPUTS

Use cases describe behaviour, not solutions

UC-002: Search & browse movies

Primary actor: API consumer (developer)

Goal: The API consumer finds the movies in the catalog that match what they are looking for (or browses the whole catalog), one page at a time and in an order they choose ...

Main flow

1. The API consumer asks the system for movies. ...
2. The system checks that the search is asked in an allowed way [BR-10].
3. The system finds every movie in the catalog that meets all the given criteria [BR-3, BR-4].

`use-cases/`

Actors, flows, exceptions, business rules. No endpoints, no tech.

`domain/`

Glossary, bounded contexts, actors, rules: the shared language.

`standards/`

Architecture, OpenAPI, errors, security, testing, formatting.

EXPERIMENT.md: same pipeline, with user stories swapped for use cases. Does the architect decide better when it isn't handed the solution?

STANDARDS & DOMAIN DOCS

How the house rules reach every plan

standards/

7 docs, ~6,300 words. Architecture, OpenAPI, errors, security, clean code, testing, formatting: **how we build**.

domain/

6 docs, ~3,900 words. Glossary, bounded contexts, actors, business rules: **what things mean**.

```
openspec/config.yaml
context: |
  … Clean Architecture, contract-
  first, Testcontainers …
  Detailed standards live in
  standards/: … Honor them.
  Domain knowledge lives in
  domain/: glossary.md, business-
  rules.md, … Use the glossary.
rules:
  specs:
    - "… on Testcontainers Postgres
      (standards/testing.md §3)."
```

Who uses it

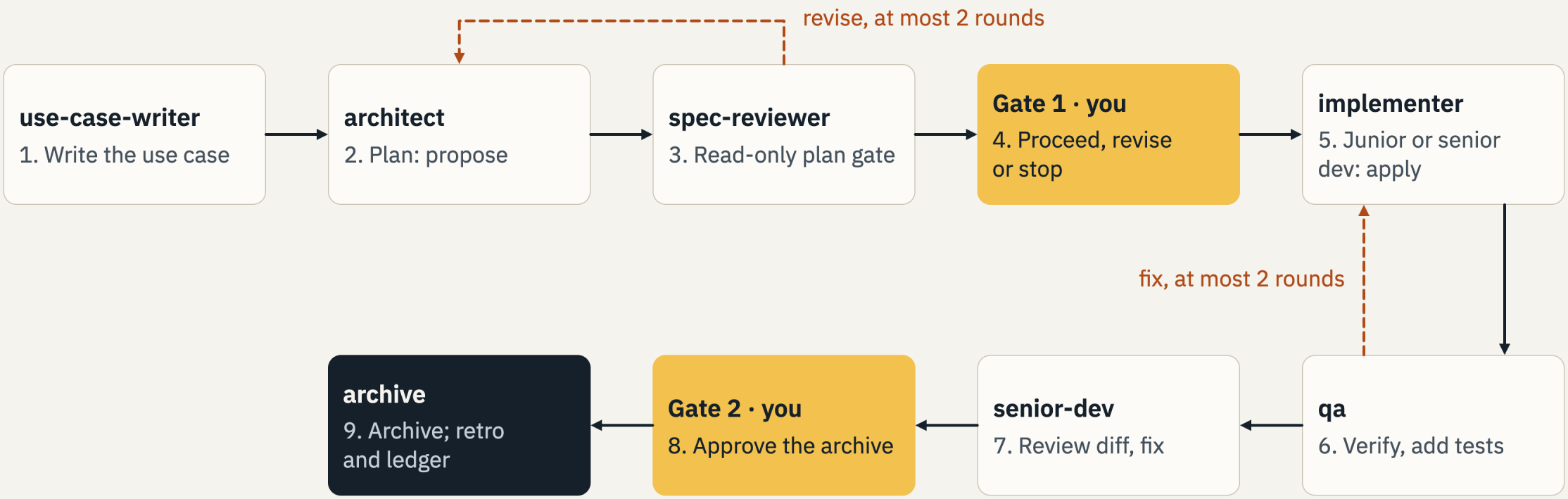
architect: gets the context on every artifact, then opens the standards and domain docs it names

gates: cite file and § on every finding

use-case writer: reads domain/ for language and rules

FROM USE CASE TO CODE

Plan, gate, build, gate, archive



HOW IT DRIVES OPENSPEC

Each phase maps to an OpenSpec verb

Phase	Who	OpenSpec	Output
Plan	architect	/opsx:propose	proposal, design, spec delta, tasks
Plan gate	spec-reviewer	validate --strict	APPROVE / REQUEST CHANGES
Gate 1	you	—	proceed / revise / stop
Implement	junior- or senior-dev	/opsx:apply	code and tests; tasks.md ticked off
Verify	qa	/opsx:verify	requirement-to-test map; READY or not
Review	senior-dev	— (reads the diff)	fixes applied; design calls handed back
Gate 2	you	—	approve the archive
Archive	orchestrator	/opsx:archive	delta folded into openspec/specs/
Retro	orchestrator	— (every run)	run record, plus the LEDGER.md update

THE REVIEWS

What the gates actually caught

spec-reviewer · UC-000 · round 2

“The architect's #8 fix followed my round-1 advice, and that advice was wrong.”

A second pass corrected the reviewer's own earlier advice.

qa · UC-000

“...all other @SpringBootTest classes ... run against the default H2 datasource too since they boot without a profile.”

Caught a silent breach of “Testcontainers, never H2”.

senior-dev · UC-002 (after QA passed)

“a query with broken percent-encoding, like ?title=%zz, turned a valid search into a 500.”

One of 5 real bugs fixed in place; 298 tests pass.

senior-dev · UC-001

“it used data.get("x").isMissingNode(), which throws a NullPointerException for an absent key instead of failing cleanly.”

A test that couldn't fail properly, rewritten.

Runs that caught something, across 3 use cases: spec-reviewer 4 of 7 · qa 2 of 3 · senior-dev 3 of 3

GUARDRAILS

Stopping runaway spend

≤ 2 rounds

per fix loop. On the third attempt, a human decides.

~10 runs

for the whole use case, then pause. Needing that many means the plan is wrong.

No fan-out

Only the orchestrator spawns agents. Agents can't spawn agents.

Token caps

16k output and 12k thinking per response, set in settings.json.

Fail closed

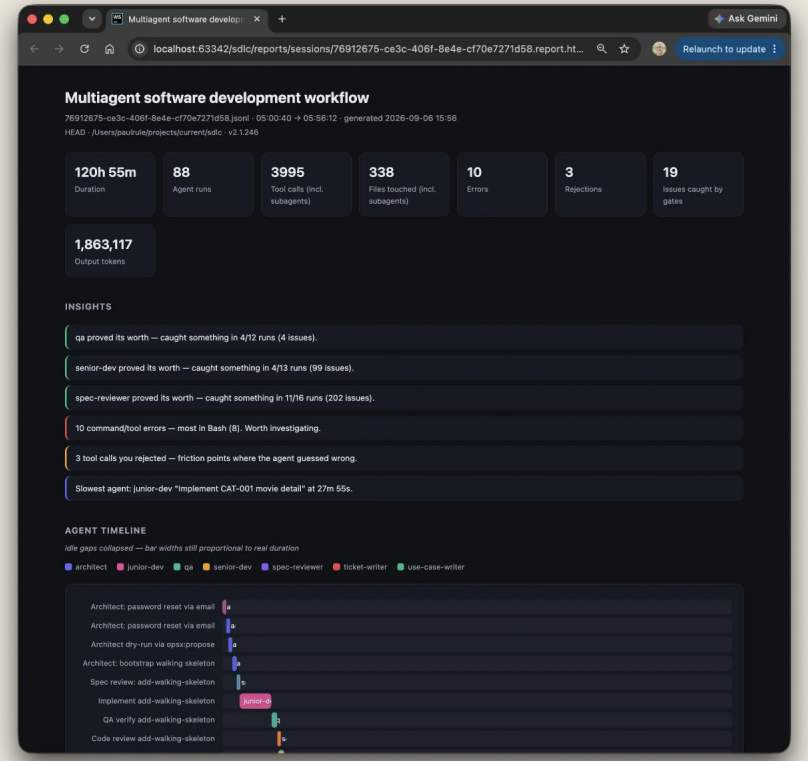
If unsure whether to spend another round, stop and ask.

No scope creep

Work beyond the plan goes to a human as a scope decision.

OBSERVABILITY

See what each run actually did



Hooks

Every tool call goes to a JSONL log, which renders to an HTML timeline. Shows what the run touched.

OpenTelemetry → Grafana

Claude Code telemetry fed to Prometheus and Loki. Shows cost, tokens and trends.

session-report skill

Agent Gantt chart, gate value, context ingestion, and --compare for A/B runs. Shows whether each agent earned its place.

SELF-IMPROVEMENT

Retrospectives turn catches into better inputs

1 · Catch

A gate finds a problem and says **why** it got through: missing standard, vague spec, prompt gap, model slip.

→

2 · Recommend

A change to an **input**: a standard, a spec rule, an agent prompt, a template. Never “be more careful”.

→

3 · Record

One dated file per run, failed runs included. A ledger counts each lesson that comes back.

→

4 · Adopt

You pick what's worth it. A PR changes the input, and the box is ticked with the PR number.

→

5 · Re-test

Every agent reads the new rule on the next run. If the finding comes back, the fix didn't work.

Fix the input, not just the code. A recurring finding means a standard, rule or prompt isn't working.

Gate agents end with a Run-log findings block · build-use-case step 7 writes the record and updates retrospectives/LEDGER.md

THE LOOP, ON THIS PROJECT

From findings to better standards

UC-002, twice

In both runs the architect planned to drop @Validated and to check query results on H2.

→

#38 · standards

Inherited @Validated allowed if pinned; H2 test scope spelled out; link building fixed.

→

#40 · four rules

QA verifies claims · malformed-input scenarios · the 500 rule · always bind values.

→

#41 · next run

QA checked 3 implementer claims against the Spring source. The new bind rule exposed the same bug on develop.

7

input changes adopted from 3 records (20 recommendations)

2

findings that kept recurring, both fixed in the standards

1

new repeat flagged: tasks paraphrase the spec's acceptance checks

Sources: retrospectives/2026-09-28-add-movie-search.md, 2026-10-03-add-movie-search.md, 2026-10-03-cleanup-uc-002-standards.md

THE COST

What one use case costs

Measure	UC-000 foundation	UC-001 details	UC-002 search
Agent runs	8	12	8
Agent working time	1h 19m	1h 56m	1h 03m
Session wall time	6h 25m	21h (overnight)	2h 09m
Output tokens	195k	133k	173k
Cache-read tokens	53.6M	35.7M	61.8M
API-equivalent cost	~$18	~$15	~$19
...of which review gates	$3.01 (17%)	$2.96 (20%)	$2.65 (14%)
Lines added: main / test	774 / 1,606	623 / 1,375	1,117 / 2,378

Session logs, orchestrator plus subagents. List prices per MTok: Opus 5.5 $4 in / $20 out, Sonnet 5 $2 / $10, cache reads $0.20, cache writes 1.25× input.

WHERE THE MONEY GOES

UC-002 cost by agent



Implementation is 64% of the cost. All three review gates together are 14%.

API-equivalent cost and active time per agent, session 05ad576e. Total ≈ $18.70.

BASELINE

UC-002 three ways, same starting commit

Measure	Plain /opsx (Opus)	Pipeline, Sonnet builds	Pipeline, all Opus
Session wall time	38 min	2h 09m	44 min
API-equivalent cost	~$6	~$19	~$9.50
Implementer tool calls	—	255	60
Human prompts	3 cmds + 1 nudge	1 cmd + 2 gates	1 cmd + 2 gates
Build; test count	green; 278	green; 298	green; 340
UC-002 flows / rules tested	8/8 · 11/11	8/8 · 11/11	8/8 · 11/11
Blind review: defects	2 low	0	1 low
Standards violations	8	7	7

Quality was a tie. **The cheap implementer is what made the pipeline expensive.**

n = 1 per arm. Blind Opus review of anonymised copies. Pipeline output tokens corrected for a CLI 2.1.288 subagent-logging gap.

HELPING OR HINDERING?

Three layers. Keep them separate.

1 · Ingestion

Was it read?

Were domain/ and standards/ opened before the first write?

2 · Utilisation

Was it used?

Does a gate finding cite the standard it enforces?

3 · Effect

Did it help?

Only comparing two runs can tell you. Reading it doesn't mean it helped.

Red flags, seen in these runs

Gate approves everything: qa in UC-001, 0 findings

One agent dominates cost: junior-dev, 45–64%

Re-plan cycles: UC-001 needed 3 spec reviews

Stalls: UC-001 junior-dev resumed 3 times

VERDICT

Is it worth it? On a strong model, often.

+$3.50
and +6 min buys independent plan and code review (all Opus)

2×
the cost with a Sonnet implementer, and 3× the time

≈ tie
on quality in blind review: 0–2 low defects in each run

The pipeline earns it when

- You want a second pair of eyes on plan and code
- You need evidence of independent review
- Runs are unattended: gates stand in for your attention
- Changes are large or risky enough to warrant a second pass

Plain /opsx is enough when

- You'll review the diff closely yourself anyway
- It's a small, well-specified feature
- It's a one-line fix: skip the spec entirely
- Nobody would read what the gates report

UC-002, one run per arm: plain /opsx ~$6 / 38 min · pipeline on Opus ~$9.50 / 44 min · pipeline with Sonnet ~$19 / 2h 09m

PART 3

A second project

Fifteen runs on Logbook: what the records showed, and what changed because of them.

LOGBOOK · 15 RUN RECORDS

The gates found problems. The inputs didn't change.

~120
recommendations across the 15 records

5
rules adopted from them, all from the first run

2 of 14
plans approved at the first spec review

Recurring finding	Records	Fixed?
Local Verify gives no single trustworthy result	13 of 15	no
A scenario part has no named test at the right layer	13 of 15	partly
The plan misses tests and specs the change breaks	7, last 5 in a row	no
Rework leaves the artifacts inconsistent	6	no

Source: Logbook's retrospective review of UC-0.1 to UC-016, 25 Oct 2026. Every record was read and its recommendations grouped by root cause.

THE FIX

A ledger counts each lesson and forces a decision

Step 0 · Brief

The open lines that apply go to the architect. Each later agent gets the lines for its phase.

→

Gate 1 · Decide

A line in 3 or more records is **due**. You adopt it in this run, or keep or close it with a reason.

→

Step 7 · Count

Each recommendation adds 1 to its line, or opens a new one. Records cite **L-n** instead of repeating text.

Id	Logbook's ledger (excerpt)	Records	Status
L-1	Verify gives one trustworthy result	13	adopted, #22
L-2	Every clause has a named test at the layer that decides it	13	adopted, #22
L-10	Questions the use-case template should ask	10	due
L-8	Find the tests a change breaks before naming tasks	7	due

Logbook started it by hand on 25 Oct and closed 7 lines in PR #22 · plugin 0.8.0 keeps it in every build-use-case run

WHO IMPLEMENTS?

The senior dev can implement. The review stays separate.

`Implementer: senior-dev`

In the profile: the project's default.

`--senior · --junior`

On build-use-case: this run only.

`Code review`

A fresh senior-dev run that sees only the change name.

Logbook, all Opus	Runs	Run cost	Agent time	Implementer calls
junior-dev	10	median $29 ($10–$90)	median 55 min	median 104
senior-dev	2	$37 · $59	58 · 95 min	130 · 155

No saving yet. Two runs on different use cases can't show one either way.

API-equivalent cost from each run's summary.json tokens, at the list prices on the cost slide · 12 Logbook runs with a session summary

LESSONS

What I'd tell a team starting out

1. **Write the requirement yourself.** The pipeline should serve it, not invent it.
2. **Invest in domain/ and standards/.** They're what the reviewers check against.
3. **Don't economise on the implementer.** Here, Sonnet made 4× the tool calls and doubled the cost.
4. **Cap the loops and fail closed.** Lots of rounds means a bad plan, not bad luck.
5. **Record, count, then decide.** A lesson that comes back 3 times needs a decision, not another record.

Next: more runs per arm (n > 1) · more senior-implementer runs · a lighter path for small changes

Questions?

OpenSpec: github.com/Fission-AI/OpenSpec

Intention Horizon: intention.vamonossoftware.com

SDLC Pipeline plugin: github.com/prule/sdlc-pipeline
SDLC Pipeline Implementation: github.com/prule/sdlc

Portfolio: portfolio.vamonossoftware.com

Blog: prule.github.io/pauls-blog/

LinkedIn: www.linkedin.com/in/paulrule/

